

Converting Images to Vector Graphics

Peter Occil

This version of the document is dated 2026-07-30.

This page concerns ways to go about remaking traditional user interface icons, which have the form of images (rectangular arrays of samples called *pixels*), as vector graphics (or geometric models), a process sometimes called *image vectorization*. The icons concerned here tend to have the following properties: ¹

- The icon's dimensions are usually 32×32 , but can range from 15×15 to 64×64 .
- The icon is drawn in a limited color palette (no more than 32 colors), ordinarily 16 colors or fewer.
- The icon may have transparent pixels, but no semitransparent (translucent) pixels.

The output should be a vector graphic with the following properties:

- A point-sampled version of the vector graphic (without antialiasing or scaling) should closely match the input image.
- The graphic is reasonably scalable, with horizontal and vertical scaling factors each from 50% through 300%. The low end is given to accommodate displays with nonsquare pixel spacing; the high end, displays with dense pixel spacing.
- Single-color line segments or curves that are one unit thick should be rendered as line or curve commands.
- Simple polygons of a single color in the input image should be rendered as such in the vector graphic.
- Size blowup compared to the image should be reasonable (for example, no more than a 2-times blowup on average). Notably, if the input image depicts a single-color polygon, the vector graphic should be a single-color polygon which often consumes less memory than the image.
- Optional: Contours recognized as arcs, ellipses, or circles should be rendered as such.
- Optional: The algorithm should recognize half-tone dithered areas as solid-colored areas.

In addition, the algorithm:

- Should not involve neural networks.
- Should not be covered by active patents or patent applications.

Algorithms that convert images to vector graphics—

- in the form of unions of shapes that cover the area between pixels (for example, *png2svg*²), or
- in the form of halftone dots that simulate the area surrounding neighboring pixels (as in newsprint or comic book halftoning),

are not within the scope of this document.

¹An image with a high number of unique colors (say, 32 or more), is hard to convert to a vector graphic without sacrificing image quality. While such images ought to be designed as vector graphics from the start, a simple upscaling and downscaling solution, such as bilinear filtering, is acceptable for such images, especially for the use case of user-interface graphics where scaling factors from 50% through 300% are expected. Limited-color images greater than 64×64 are also of interest and occur among user-interface graphics (such as wizard graphics in Windows 95), but are not the main priority; indeed, the larger the image size, the more time the conversion to a vector graphic is expected to take.

²<https://github.com/xyproto/png2svg>

1 Existing Algorithms

There is an algorithm by Kopf and Lischinski (“**Depixelizing Pixel Art**”³, *ACM Transactions on Graphics* 30(4), 2011) that tackles the task of generating vector graphics from “pixel-art” images, including those of the kind at issue here. However:

1. The algorithm generates curved shapes even if they originate from 1-unit thick line segments or curves.
2. The algorithm doesn’t generate polygons even in cases where that shape is best.
3. The algorithm runs slowly: tens of seconds to generate a vector graphic from a 64×64 icon.
4. The resulting vector graphic does not have a reasonable size compared to the input image.
5. I don’t know whether the algorithm is covered by any pending or active patents.

Potrace version 1.16 (2019) likewise suffers from points 1 and 2, and also supports only black-and-white opaque images and not color ones.

Algorithms designed for scaling images, such as Eric Johnston’s EPX, Derek Liauw Kie Fa’s 2xSaI, Maxim Stepin’s Hqx, and Andrea Mazzoleni’s Scale2x, have no known adaptation for converting an image to a vector graphic. In any case, none of the algorithms mentioned renders 1-unit-thick lines as vector line commands.

A **related project**⁴ involves drawing enlarged versions of a vector graphic defined essentially by 1-unit-thick lines and by flood fills rather than polygons.

2 Example

Take the following image:



Figure 1: **Diamond image**

The desired vector graphic should have the following commands in this order:

1. Polygon, filled with white, connecting the points (1.5, 15.5), (15.5, 1.5), (29.5, 15.5), (15.5, 29.5). (The coordinates adjust by 0.5 because of the placement of pixels at half-integer coordinates in SVG, for example.)
2. 1-unit-thick polyline (sequence of line segments), colored black, connecting the points in the foregoing polygon. Alternatively, that polygon is stroked with a 1-unit-thick black outline in addition to being filled.⁵

In SVG⁶, the desired vector graphic looks like:



Figure 2: **Diamond vector graphic**

³<https://johanneskopf.de/publications/pixelart/>

⁴<https://github.com/eviltrout/agi-upscale>

⁵In SVG, the stroke would also be marked with the `vector-effect:non-scaling-stroke` style, so that the stroke looks 1 unit thick regardless of scaling.

⁶<https://peteroupc.github.io/svg.html>

3 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under the **Unlicense**⁷.

4 Notes

⁷<https://unlicense.org>